

TP3 Java Yassine Hamrouni

Exercice 1 - Livre

1- Déclarer un record Book avec les composants title et author.

```
public record Book(author, title){}
```

2- Puis essayer le code suivant dans une méthode main du record Book.

```
var book = new Book("Da Vinci Code", "Dan Brown");  
System.out.println(book.title+' '+book.author);
```

Ce code ci-dessus permet de créer à partir du record Book, un objet Book nommé “book”. book.title sert à accéder au titre de l’objet book, idem pour book.author.

3- Créer une classe Main (dans un fichier Main.java) et déplacer le main de la classe Book dans la classe Main.

```
public class Main{  
    public static void main(String[] args){  
        var book = new Book("Da Vinci Code", "Dan Brown");  
        System.out.println(book.author+" "+book.title);  
    }  
}
```

Quel est le problème ? Comment peut-on le corriger ?

Le problème rencontré est le fait que les attributs de l’objet book sont privés. Pour y remédier, il suffit de prendre les getters définis par défaut dans un Record.

```
System.out.println(book.author()+" "+book.title());
```

4- On peut remarquer que le code permet de créer des livres ayant un titre ou un auteur null.

Comment faire pour éviter ce problème sachant qu’il existe une méthode static requireNonNull dans la classe java.util.Objects.

```
Objects.requireNonNull(author, "L'auteur ne doit pas être null.");  
Objects.requireNonNull(title, "Le titre ne doit pas être null.");
```

5- Commenter le code précédent et utiliser un constructor compact à la place.

```
public Book(){
    Objects.requireNonNull(author, "L'auteur ne doit pas être null.");
    Objects.requireNonNull(title, "Le titre ne doit pas être null.");
}
```

6- Écrire un autre constructeur qui prend juste un titre et pas d'auteur et ajouter un code de test dans le main.

On initialisera le champ author avec "" dans ce cas.

```
public Book(String title) {
    this("<no author>", title);
    Objects.requireNonNull(title, "Le titre ne doit pas être null.");
}
```

7- Comment le compilateur fait-il pour savoir quel constructeur appeler ?

Quand on crée un objet en Java et on dit new, le compilateur regarde les infos qu'on a mis entre les parenthèses pour décider quel constructeur on veut utiliser. Si on donne juste un titre de livre, Java va chercher un constructeur dans la classe qui attend un titre.

8- On souhaite maintenant pouvoir changer le titre d'un livre déjà existant en utilisant une méthode nommée withTitle qui prend en paramètre le nouveau titre.

Pourquoi le code suivant ne marche pas ?

Car on n'est pas censé modifier les attributs d'un record.

Comment faire alors ?

```
public static void main(String[] args){
    var originalBook = new Book("J.K. Rowling", "Harry Potter");
    var newBook = originalBook.withTitle("Harry Potter and the Chamber of Secrets");

    System.out.println("Original: " + originalBook.title());
    System.out.println("New: " + newBook.title());
}
```

Exercice 2 - Liberté, Égalité, toString

1- Qu'affiche le code ci-dessous ?

```
var b1 = new Book("Da Java Code", "Duke Brown");
var b2 = b1;
var b3 = new Book("Da Java Code", "Duke Brown");

System.out.println(b1 == b2);
System.out.println(b1 == b3);
```

`System.out.println(b1 == b2);` affiche `true`, parce que `b1` et `b2` pointent vers la même instance en mémoire. L'opérateur `==` compare les références (les adresses mémoire) des objets, et dans ce cas, les références sont identiques.

`System.out.println(b1 == b3);` affiche `false`, parce que même si `b1` et `b3` représentent des objets avec les mêmes valeurs de champs (titre et auteur), ils sont deux instances distinctes situées à des adresses mémoire différentes. L'opérateur `==` compare les références, pas le contenu des objets, donc cette comparaison renvoie `false`.

2- Comment faire pour tester si deux objets ont le même contenu ?

```
var b1 = new Book("Da Java Code", "Duke Brown");
var b2 = b1;
var b3 = new Book("Da Java Code", "Duke Brown");

System.out.println(b1.equals(b2));
System.out.println(b1.equals(b3));
```

3- Écrire une méthode `isFromTheSameAuthor()` qui renvoie vrai si deux livres sont du même auteur.

Et vérifier avec les deux livres suivants :

```
var book1 = new Book("Da Vinci Code", "Dan Brown");
var book2 = new Book("Angels & Demons", new String("Dan Brown"));
```

Réponse:

```
public boolean isFromTheSameAuthor(Book b){
    return this.author.equals(b.author);
}
```

4 - Comment faire pour que le code suivant

```
var javaBook = new Book("Da Java Code", "Duke Brown");
System.out.println(javaBook);
```

Affiche

Da Java Code by Duke Brown

5- Utiliser l'annotation `@Override` (`java.lang.Override`) sur la méthode ajoutée à `Book`.

6- A quoi sert l'annotation `@Override` ?

Réponse 4, 5, 6: Pour afficher cela, Il faut réécrire avec la notation `@Override` la méthode `toString` présent sur tout `Object`, qui retourne par défaut, l'adresse de l'objet.

```
@Override
public String toString(){
    return this.title+" by "+this.author;
}
```

Exercice 3 - Liberté, equals, Fraternité

```
public class Book2 {
    private final String title;
    private final String author;

    public Book2(String title, String author) {
        this.title = title;
        this.author = author;
    }

    public static void main(String[] args) {
        var book1 = new Book2("Da Vinci Code", "Dan Brown");
        var book2 = new Book2("Da Vinci Code", "Dan Brown");
        System.out.println(book1.equals(book2));
    }
}
```

1- Quel est le problème ?

Le problème avec le code présenté est que la méthode `equals` par défaut de la classe `Object` est utilisée pour comparer les instances de `Book2`. Cette méthode compare les références (adresses mémoire) des objets, et non pas leur contenu. Puisque `book1` et `book2` sont deux instances distinctes (même si elles ont les mêmes valeurs pour `title` et

author), la comparaison avec equals retournera false. Ce n'est pas le comportement attendu, car nous voulons comparer les objets basés sur le contenu de leurs attributs, et non pas sur leurs références.

2- Comment corriger le problème si on s'entête à utiliser une classe ?

Nous devons surcharger (override) la méthode equals dans notre classe Book2. En surchargeant cette méthode, nous pouvons définir notre propre logique de comparaison qui vérifiera l'égalité des titres et des auteurs des deux livres.

```
@Override
public boolean equals(Object obj) {
    if (obj instanceof Book2) {
        Book2 otherBook = (Book2) obj;
        return this.title.equals(otherBook.title) && this.author.equals(otherBook.author);
    }
    return false;
}
```

Exercice 4

1- Écrire une méthode swap qui échange les valeurs de deux cases d'un tableau :

```
public static void swap(int[] array, int index1, int index2){
    var size = array.length;
    if(index1 < 0 || index1 > size || index2 < 0 || index2 > size || index1 > index2){
        System.out.println("Merci de mettre des indices correctes.");
    }
    else{
        if(index1 != index2){
            var tmp = array[index1];
            array[index1] = array[index2];
            array[index2] = tmp;
        }
    }
}
```

2- Écrire une méthode indexOfMin qui renvoie l'indice de la valeur minimale d'un tableau.

```
public static int indexOfMin(int[] tableau) {
    if (tableau == null || tableau.length == 0){
        throw new IllegalArgumentException("Le tableau ne peut pas être vide ou nul.");
    }
}
```

```

    int indiceMin = 0;
    for (int i = 1; i < tableau.length; i++) {
        if (tableau[i] < tableau[indiceMin]) {
            indiceMin = i;
        }
    }
    return indiceMin;
}

```

3- Modifier la méthode `indexOfMin` en ajoutant deux indices indiquant que l'on cherche l'indice du minimum, non pas sur tout le tableau, mais sur la partie de tableau entre ces deux indices (le premier inclus, le deuxième exclu).

```

public static int indexOfMin(int[] tableau, int start, int end) {
    if (tableau == null || tableau.length == 0 || start < 0 || end > tableau.length || start > end) {
        throw new IllegalArgumentException("Paramètres invalides.");
    }

    int indiceMin = start;
    for (int i = start + 1; i < end; i++) {
        if (tableau[i] < tableau[indiceMin]) {
            indiceMin = i;
        }
    }
    return indiceMin;
}

```

4- Écrire la méthode `sort` qui prend un tableau d'entiers en paramètre et qui trie celui-ci en utilisant pour cela les méthodes `indexOfMin` et `swap`.

```

public static void sort(int[] array){
    if (array == null || array.length == 0){
        throw new IllegalArgumentException("Le tableau ne peut pas être vide ou nul.");
    }

    for (int i = 0; i < array.length - 1; i++) {
        int indiceMin = indexOfMin(array, i, array.length);
        swap(array, i, indiceMin);
    }
}

```